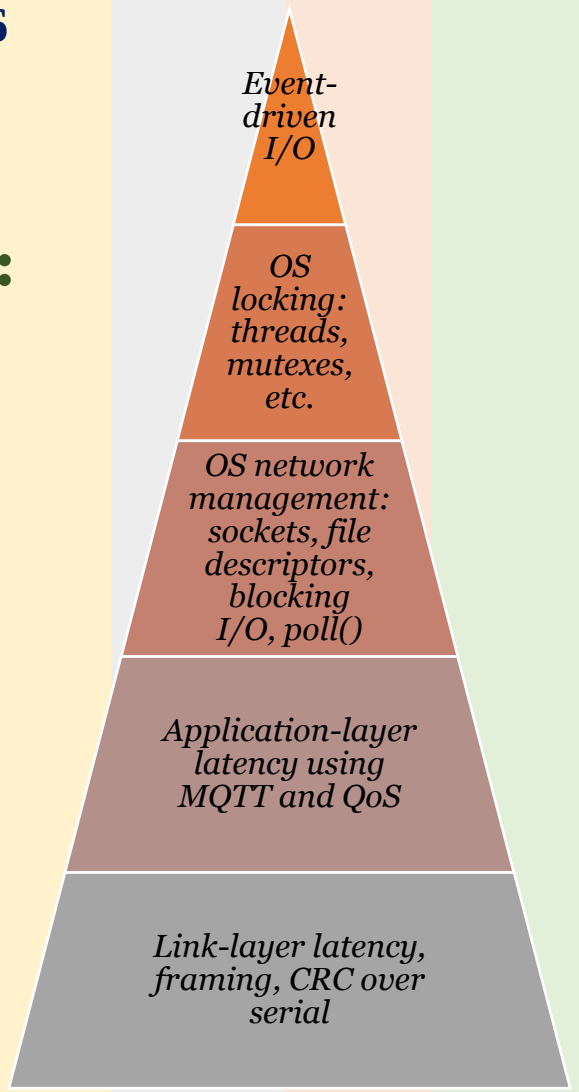
School of Information Science and Engineering

International Crash Course and Practical Training Week

Lanzhou University, Gansu, China – Mon-Friday, July 20-24, 2026

Network and OS Foundations for Safe Robotic Actuation: *Latency, Reliability, and Event-Driven Control in Industrial Hair-Cutting Robots*

Audience Level: Advanced Undergraduate / Early Post-Graduate



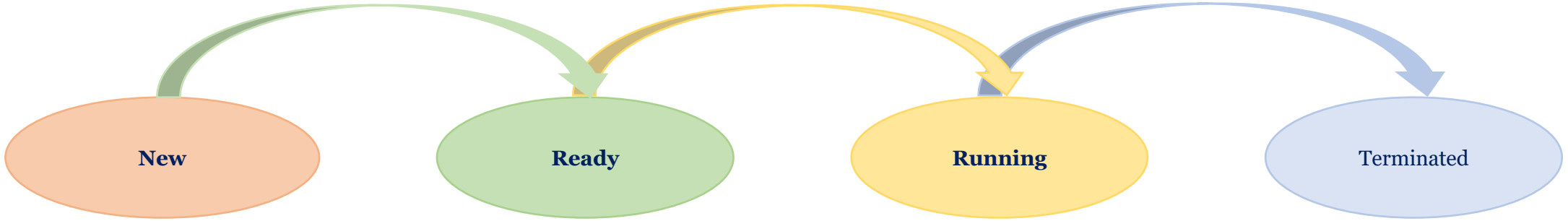
Prepared and Presented by

Prof. Dr. Mohammed Aquil Mirza

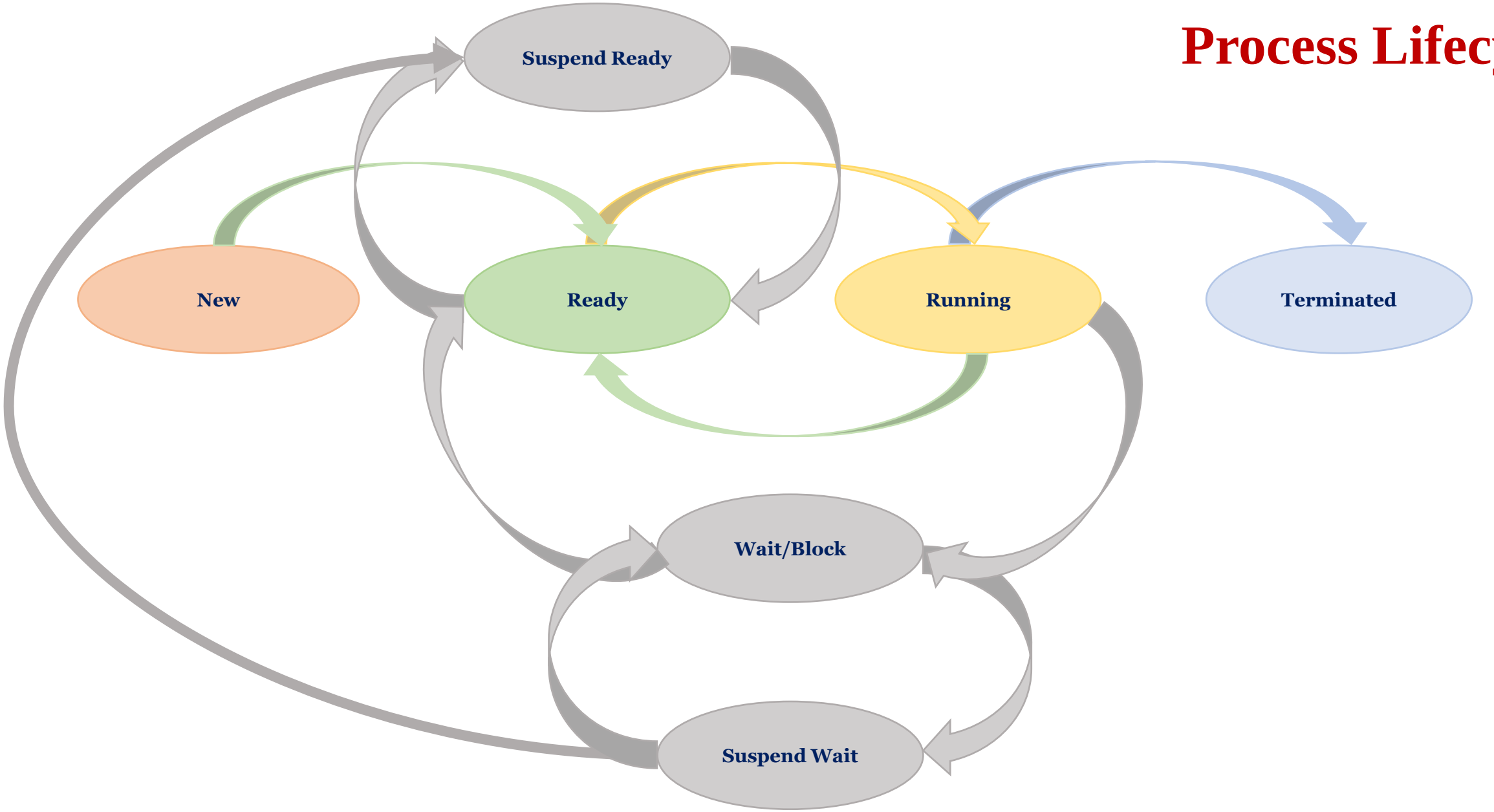
Fellow IESE *(UK),* ***Fellow IETE*** *(India),* ***Fellow HKIE*** *(Hong Kong SAR),*
Fellow AAIA *(Singapore),* ***Senior Member IEEE*** *(USA),* ***Life Member ACM*** *(USA)*

Dean & Professor*, School of Low Altitude and Digital Intelligence,* ***L****iaon****i****ng* ***T****echnical* ***U****niversity (****LNTU****), China*
Senior Lecturer*, Department of Computing,* ***T****he* ***H****ong* ***K****ong* ***P****olytechnic* ***U****niversity (****PolyU****), Hong Kong S.A.R.*

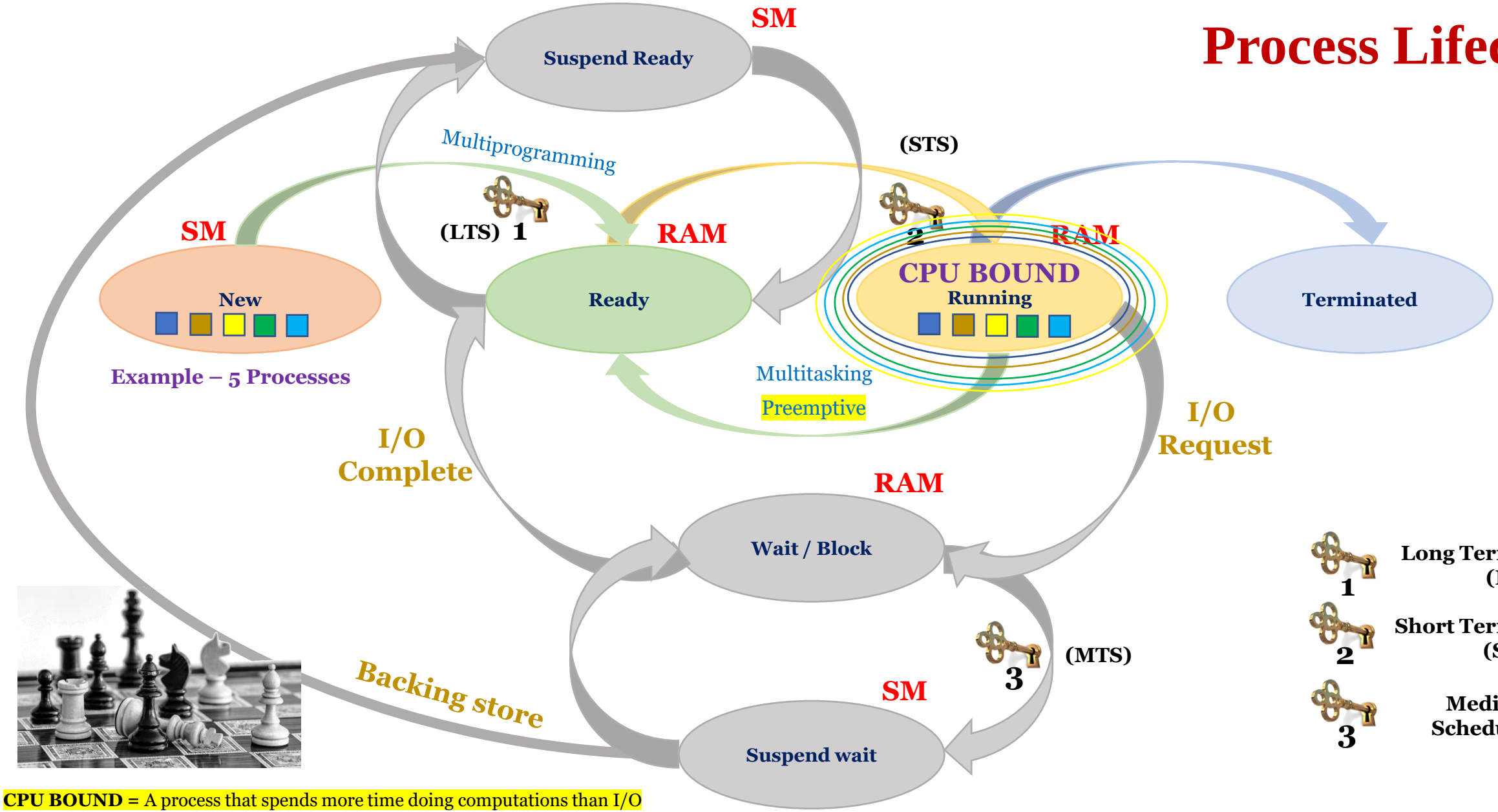
Process Lifecycle



Process Lifecycle

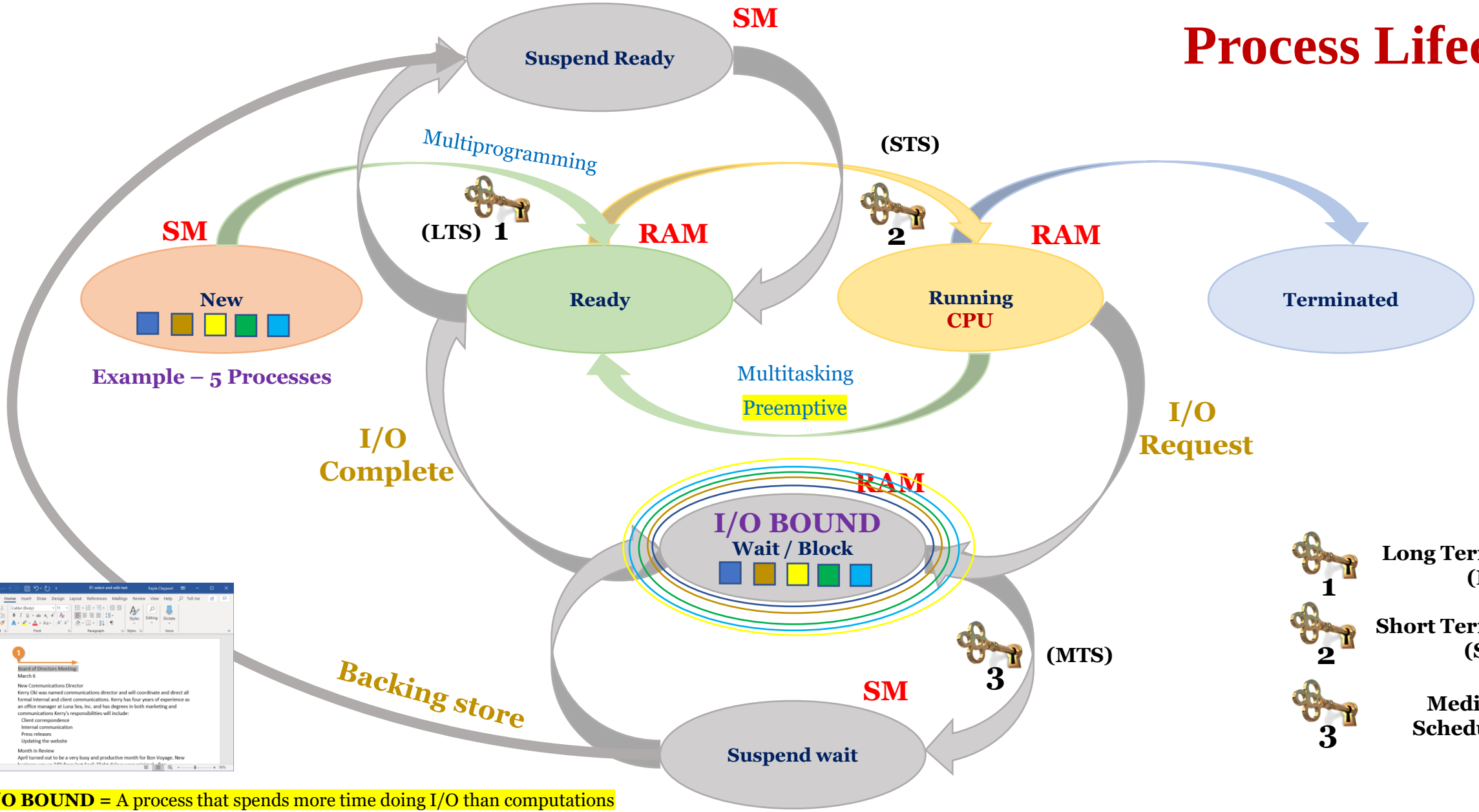


Process Lifecycle



CPU BOUND = A process that spends more time doing computations than I/O

Process Lifecycle



1 Long Term Schedular (LTS)

2 Short Term Schedular (STS)

3 Medium Term Schedular (MTS)

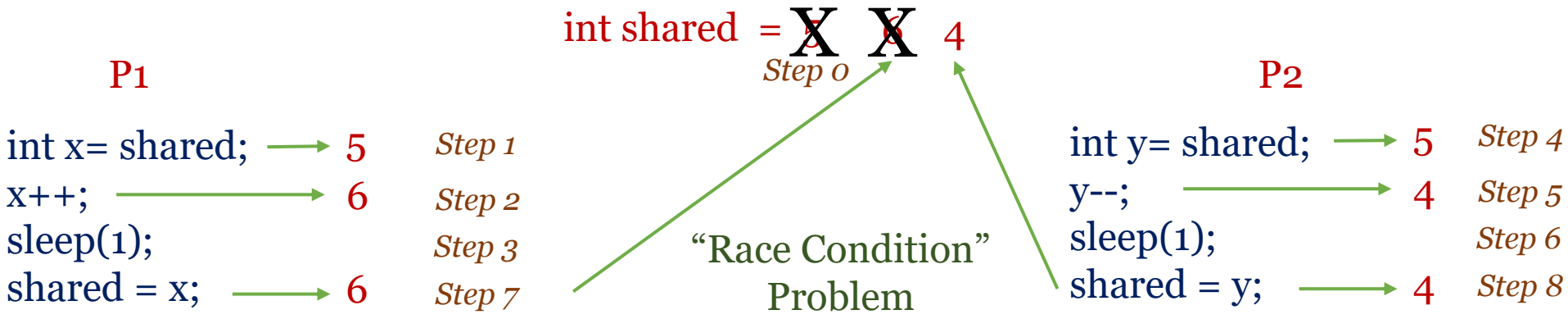
I/O BOUND = A process that spends more time doing I/O than computations

Cooperative Processes
(The need for synchronization)

RECALL
Lecture 04 Animated Slides (32 – 60)

- Two types –
  - Cooperative process (execution of one process (**may**) effect other processes)
    - Cooperative as they share either a **variable**, **memory**, **code** or **resources** (CPU, printer, scanner, etc.)
    - *Example – 1000 people access Cathey Pacific Server for Booking Flights*
    - If these processes are not synchronized properly they can create a problem!!!

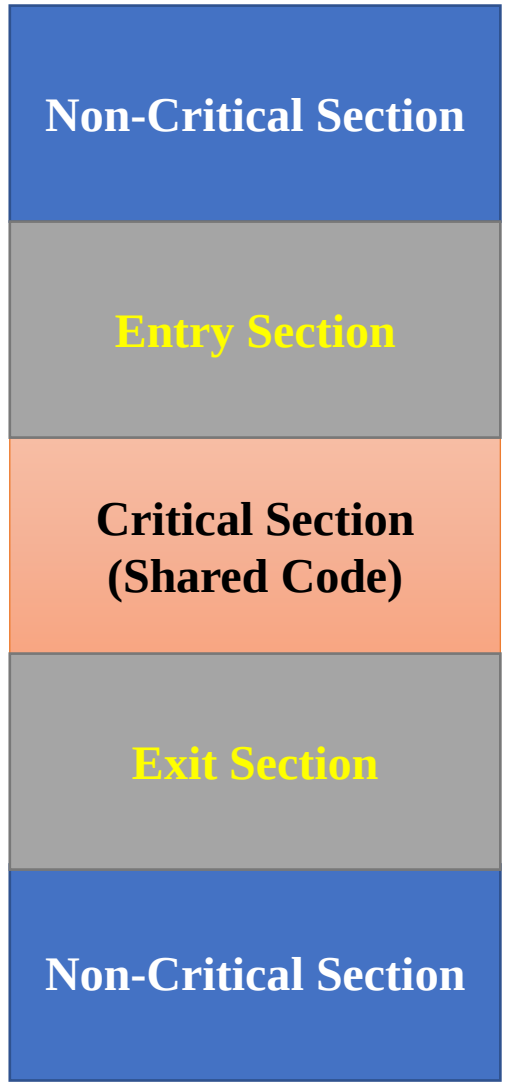
Consider *shared* is a global var and the processes *P1* and *P2* have the limitation of uni-processor



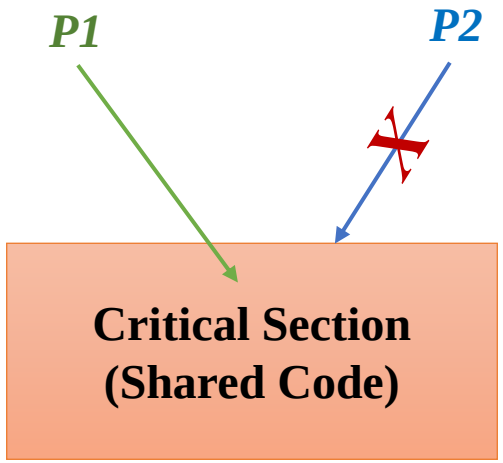
Critical Section Problem

```
int x= shared;
x++;
sleep(1);
shared = x;
```

```
int y= shared;
y--;
sleep(1);
shared = y;
```

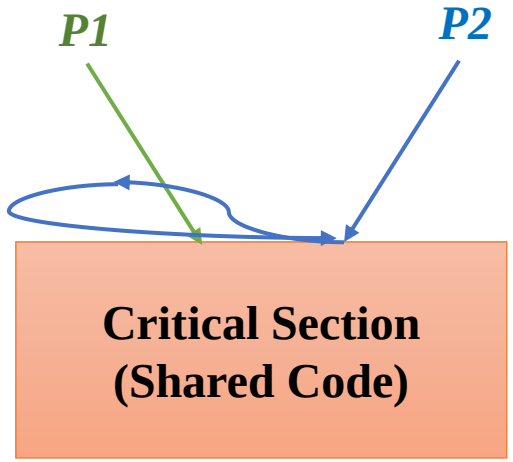


Critical Section Problem



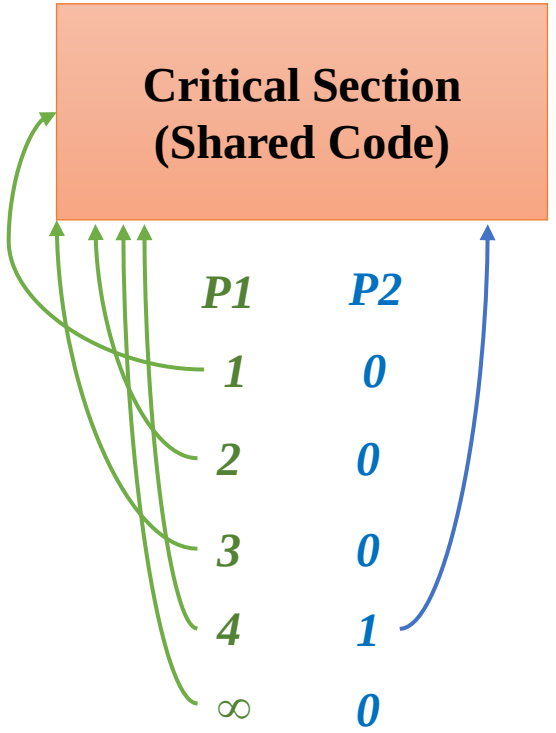
P2 cannot enter CS as P1 is using it

1. MUTUAL EXCLUSION



P2 entry code is stopping P1 from entering CS

2. PROGRESS



P1 have used CS infinite time and P2 has used only once/twice or for a very very small number

3. BOUNDED WAIT

Primary Rules

Secondary Rules

Use of Semaphores

- Semaphores could be used to solve other synchronization problems.
  - Producer/consumer (or bounded buffer) problem.
  - Dining-philosophers problem.
  - Sleeping-barber problem.
  - Cigarette-smoker problem.
  - Readers/writers problem.
  - Society-room problem.
- Let us use semaphores to solve the producer / consumer problem (bounded buffer problem) with *N* buffers.
  - Define a semaphore to implement critical section.
    - Semaphore mutex = 1;

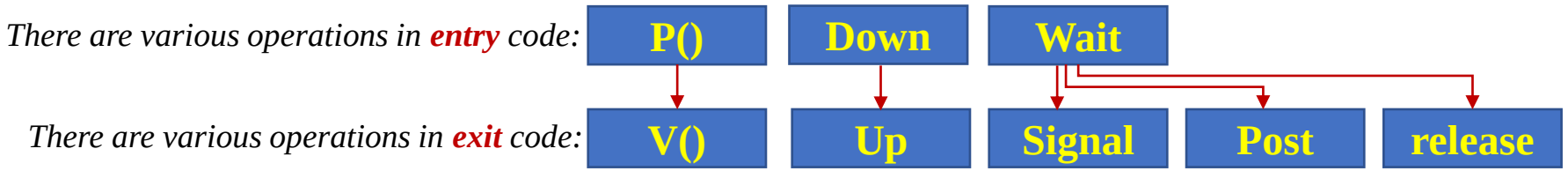
Semaphores

- We want a synchronization tool that does not require busy waiting.
- Instead, the OS could suspend the process waiting to enter the critical section.
- The most common tool is the *semaphore*.
  - A semaphore *S* is an integer variable.
  - It provides two standard operations: P(*S*) and V(*S*).
  - Originally, semaphore means flag signal
- P(*S*) means that the process will wait on *S* until *S* becomes positive.

  while (*S* <= 0) do { }; // wait until *S* becomes positive

  S = *S* – *1* ;
- V(*S*) means that *S* will be increased by one and the process waiting on *S* could proceed.

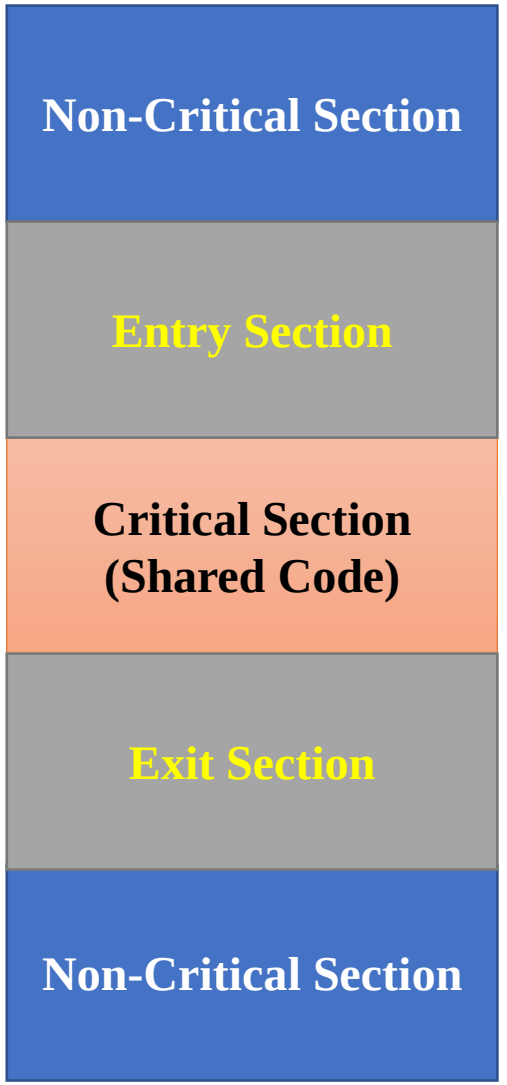
  S = *S* + 1;



Semaphores

- Two types of semaphores
  - *Binary semaphores*: integer value can only be 0 or 1.
  - *Counting semaphores*: integer value can be any positive number, negative value or zero.
- Binary semaphores are easy to implement.
  - They are also called *mutex* locks (they look like locks and they help to ensure mutual exclusion or *mutex*).
  - Providing critical section with semaphore **S** initialized to **1**.

```
while (true) do {
    P(S);                  // entry section
    < critical section >
    V(S);                  // exit section
    < remainder section >
}
```



Semaphore

Counting (**– ∞ to + ∞**)

Binary (0, 1)

Semaphore is a integer variable which is used in mutual exclusive manner by various concurrent cooperative processes in order to achieve synchronization

Semaphore is a tool/method to prevent race condition (loss of data, deadlock, etc.)

```
Down(semaphore S)
{
    S.value = S.value – 1
    if(S.value < 0)
    {
        put process (PCB)
              in suspend list
        sleep();
    }
    else
        return;
```

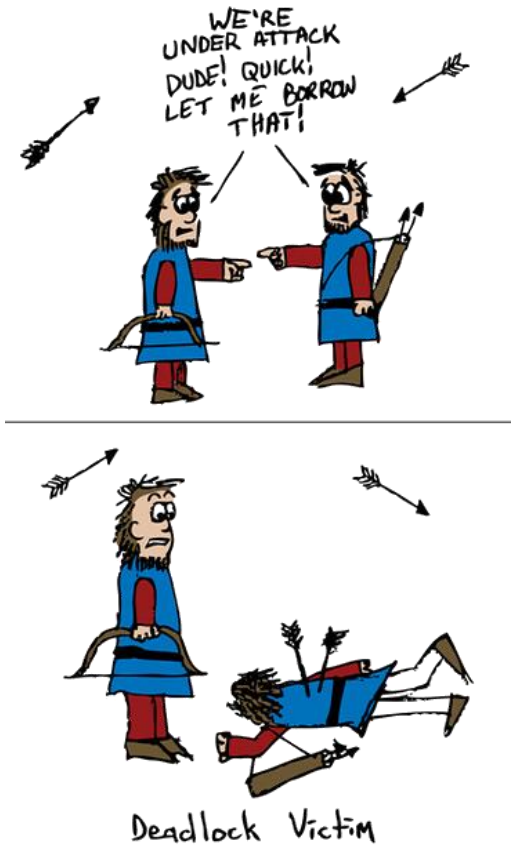
```
Up(semaphore S)
{
    S.value = S.value + 1
    if(S.value <= 0)
    {
        Select a process
              from suspend list
        wakeup();
    }
}
```

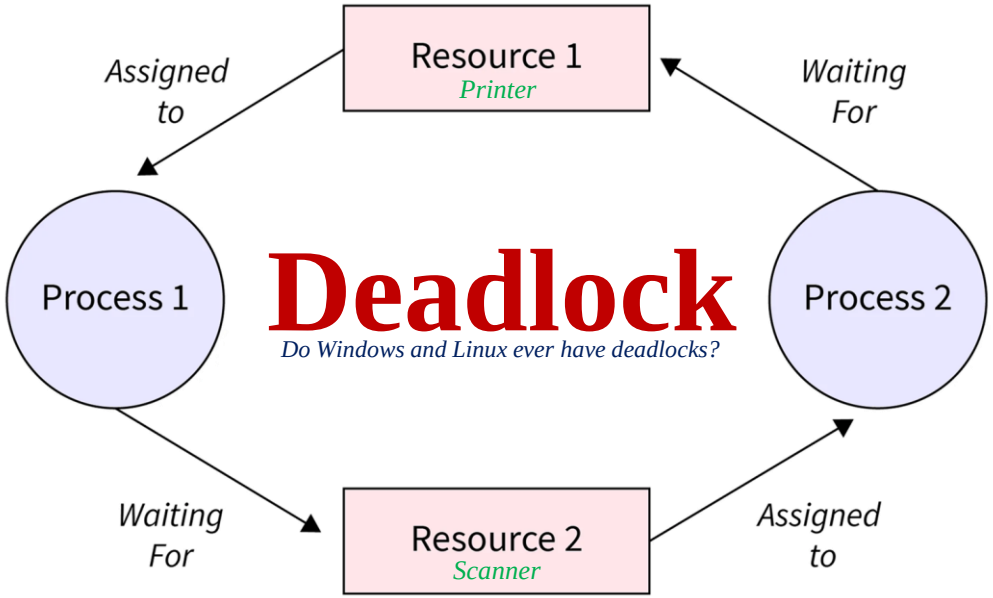
```
Process
{
    Non Critical Section
    Entry Section
    Critical Section
    Exit Section
    Non Critical Section
}
```

*There are various operations in **entry** code:* **P()** **Down** **Wait**

*There are various operations in **exit** code:* **V()** **Up** **Signal** **Post** **release**

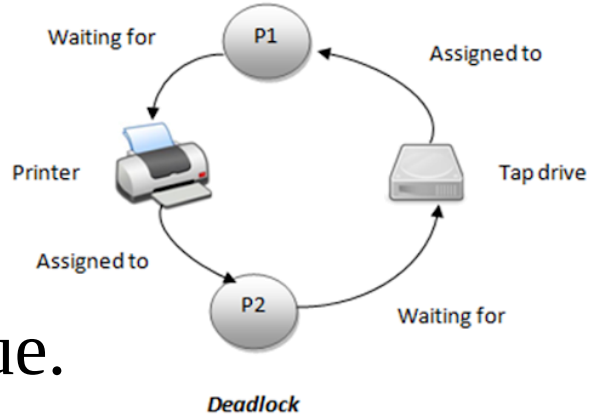
Historically, sem wait() was called P() by Dijkstra and sem post() called V(). These shortened forms come from Dutch words; interestingly, which Dutch words they supposedly derive from has changed over time. Originally, P() came from "passering" (to pass) and V() from "vrijgave" (release); later, Dijkstra wrote P() was from "prolaag", a contraction of "probeer" (Dutch for "try") and "verlaag" ("decrease"), and V() from "verhoog" which means "increase". Sometimes, people call them down and up. Use the Dutch versions to impress your friends, or confuse them, or both. See https://news.ycombinator.com/ item?id=8761539) for details.


WE'RE UNDER ATTACK DUDE! QUICK! LET ME BORROW THAT!
Deadlock Victim


Assigned to
Resource 1
Printer
Waiting For
Process 1
Deadlock
Do Windows and Linux ever have deadlocks?
Process 2
Waiting For
Resource 2
Scanner
Assigned to

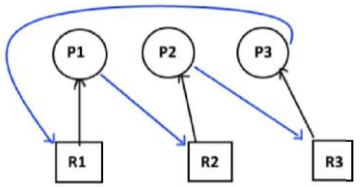

Buy me an ice-cream FIRST then I will do homework
If you finish homework FIRST I will buy you an ice-cream

Deadlock Characterization



- Deadlock can only occur if all four *necessary conditions* are true.
  - *Mutual exclusion*
    - Only one process at a time can use a resource.
  - *Hold and wait*
    - A process holding at least one resource is waiting to acquire additional resources held by other processes.
  - *No preemption*
    - A resource can be released only voluntarily by the process holding it, after that process completes its task.
  - *Circular wait*
    - There is a chain of waiting processes $\{P_1, P_2, \ldots, P_n, P_1\}$ such that P_1 is waiting for a resource that is held by P_2, P_2 is waiting for a resource that is held by P_3, …, P_{n-1} is waiting for a resource that is held by P_n, and P_n is waiting for a resource that is held by P_1.

Which of the two events are mutually exclusive (only one event can happen after another event)?
a) Two students uploading assignment on Blackboard
b) Two students printing assignment on the single printer



Q & A

Thank you. I welcome your questions!